Regularization

Improving Generalization

Firuz Kamalov

MAI 604: Deep Learning

March 28, 2026

Lecture Outline

Introduction to Regularization

- **Generalization gap:** The performance discrepancy between the training data and the test data (overfitting).
- **Regularization:** A family of techniques used to reduce the generalization gap between training and test performance.
- **Categories of Regularization:**
  1. *Explicit regularization:* Adding mathematical penalty terms to the loss function.
  2. *Implicit regularization:* The optimization algorithm (SGD) inherently favors certain solutions.
  3. *Heuristics:* Early stopping, ensembling, dropout, transfer learning, etc.

Explicit Regularization

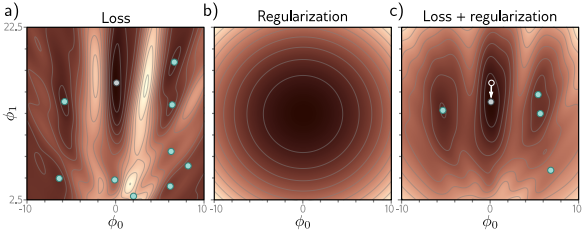
- Involves adding an explicit term to the loss function to favor certain parameter choices.
- **Original loss minimization:**

$$\hat{\phi} = \text{argmin}_{\phi} \sum_{i=1}^{I} \ell_i[\mathbf{x}_i, \mathbf{y}_i]$$

- **Regularized loss minimization:**

$$\hat{\phi} = \text{argmin}_{\phi} \left[\sum_{i=1}^{I} \ell_i[\mathbf{x}_i, \mathbf{y}_i] + \lambda \cdot g[\phi] \right]$$

- λ is a positive scalar that controls the relative contribution of the regularization term.
- $g[\phi]$ penalizes less preferred parameters.



Probabilistic Interpretation

- Regularization can be viewed from a probabilistic perspective.
- The standard loss function corresponds to the **maximum likelihood** criterion:

$$\hat{\phi} = \text{argmax}_{\phi} \left[\prod_{i=1}^{I} Pr(\mathbf{y}_i | \mathbf{x}_i, \phi) \right]$$

- The regularization term acts as a prior distribution $Pr(\phi)$ over parameters, leading to the maximum a posteriori criterion:

$$\hat{\phi} = \text{argmax}_{\phi} \left[\prod_{i=1}^{I} Pr(\mathbf{y}_i | \mathbf{x}_i, \phi) Pr(\phi) \right]$$

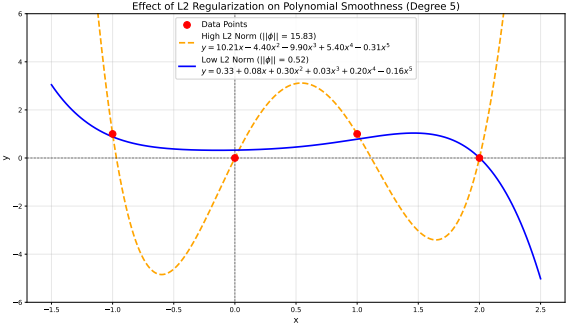
- Applying the negative log-likelihood maps this to explicit regularization: $\lambda \cdot g[\phi] = -\log[Pr(\phi)]$.

L2 Regularization

- The most commonly used regularization term, penalizing the sum of squares of parameters (ridge regression).

$$\hat{\phi} = \text{argmin}_{\phi} \left[\sum_{i=1}^{I} \ell_i[\mathbf{x}_i, \mathbf{y}_i] + \lambda \sum_{j} \phi_j^2 \right]$$

- **Effect:** Encourages smaller weights, leading to smoother output functions.



The Role of λ in L2 Regularization

- The hyperparameter λ controls the trade-off between strict adherence to the training data and the desire for model smoothness.

$$\hat{\phi} = \text{argmin}_{\phi} \left[\sum_{i=1}^{I} \ell_i[\mathbf{x}_i, \mathbf{y}_i] + \lambda \sum_{j} \phi_j^2 \right]$$

- If λ is too small, the model focuses almost entirely on minimizing training error. It passes exactly through data points (overfitting/variance).
- If λ is too large, the regularization penalty dominates the loss. The function becomes overly smooth and less accurate (underfitting/bias).

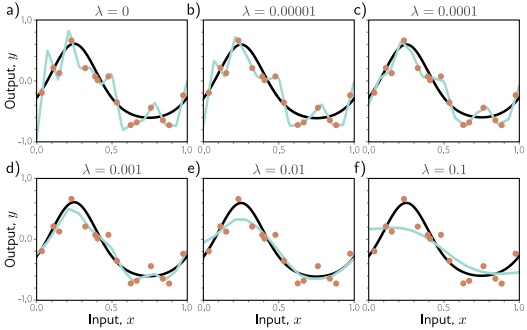


Figure: The optimal λ is between 0.0001 and 0.001.

Implicit Regularization in Gradient Descent

- Neither GD nor SGD moves neutrally to the minimum; they prefer certain solutions. This is **implicit regularization**.
- Discretizing the GD causes the path to diverge from continuous gradient descent:

$$\phi_{t+1} = \phi_t - \alpha \frac{\partial L[\phi]}{\partial \phi}$$

- Described by adding a penalty for the squared gradient to the original (continuous) loss:

$$\tilde{L}_{GD}[\phi] = L[\phi] + \frac{\alpha}{4} \left\| \frac{\partial L}{\partial \phi} \right\|^2$$

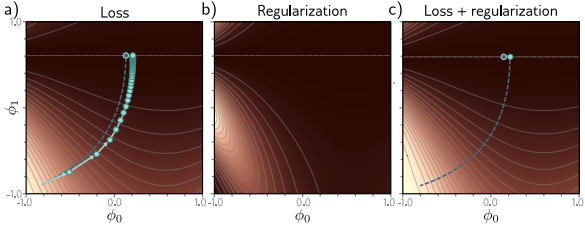


Figure: The trajectory is repelled from steep areas with large gradient norms.

Implicit Regularization in Stochastic Gradient Descent

- SGD introduces randomness. Its modified continuous loss adds a term penalizing the **variance of the batch gradients**:

$$\tilde{L}_{SGD}[\phi] \approx L[\phi] + \frac{\alpha}{4}\left\|\frac{\partial L}{\partial \phi}\right\|^2 + \frac{\alpha}{4B}\sum_{b=1}^{B}\left\|\frac{\partial L_b}{\partial \phi} - \frac{\partial L}{\partial \phi}\right\|^2$$

- SGD inherently favors regions where gradients are stable (where all batches agree on the slope).
- Forces models to fit *all* data well rather than fitting some data extremely well and other data poorly.

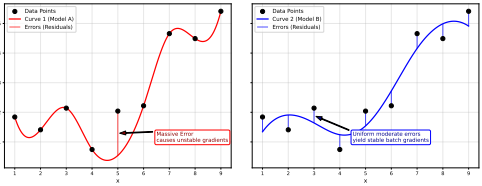
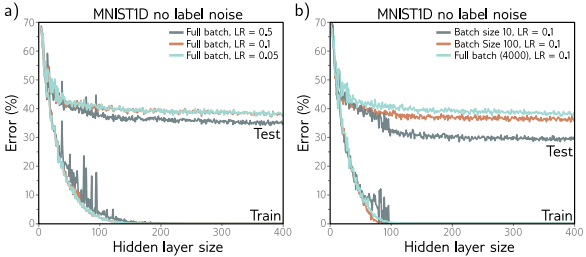


Figure: Model A (left): High variance in errors; MSE=0.25, Var=0.5. Model B (right): Low variance in errors; MSE=0.25, Var=0.

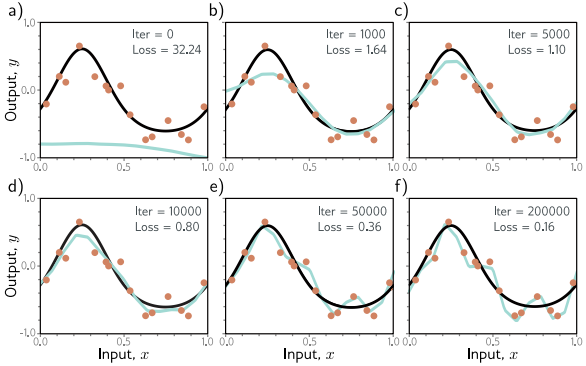
Effect of Learning Rate and Batch Size

- **Learning Rate (α):** A larger learning rate amplifies the penalty on gradient magnitude and variance, enhancing implicit regularization.
- **Batch Size ($|B|$):** Smaller batch sizes increase batch variance, strengthening the preference for stable gradient regions.
- **Observation:** Full batch or large batch GD generally generalizes worse than *SGD with appropriately large step sizes and smaller batches.*



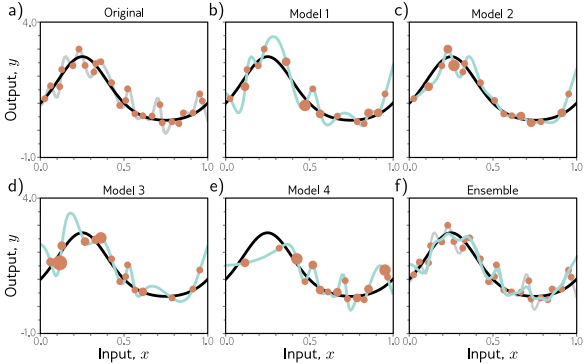
Early Stopping

- **Idea:** Stop the training procedure before the loss fully converges.
- The network captures the coarse underlying function shape early on, but needs more time to overfit to the noise.
- Since weights are initialized to small values, terminating early prevents weights from growing large (similar effect to **L2**).
- Practice: Monitor performance on a *validation set* every T iterations and store the model that performs best.



Ensembling

- **Idea:** Build multiple models and average their predictions (mean, median, or majority vote).
- Errors made by individual models are often independent and will cancel out.
- **Creating diversity:**
  - Train models using different random *initializations*.
  - Bagging (Bootstrap aggregating): *Resample* the training dataset with replacement.
  - Use different *hyperparameters* or model families.
- **Result:** A smoother, more robust function.



Ensemble Method: Numerical Example

Problem: Predict the output for input $x = 2$ using an ensemble of 3 networks.

Model 1

$$W^{(1)} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, b^{(1)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} 2 & 1 \end{bmatrix}, b^{(2)} = -1$$

$$z^{(1)} = \begin{bmatrix} 1 \\ -1 \end{bmatrix} (2) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$$

$$h = \text{ReLU}(z^{(1)}) = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$\mathbf{y_1} = \begin{bmatrix} 2 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 0 \end{bmatrix} - 1 = 3$$

Model 2

$$W^{(1)} = \begin{bmatrix} 0.5 \\ 2 \end{bmatrix}, b^{(1)} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} -1 & 0.5 \end{bmatrix}, b^{(2)} = 0$$

$$z^{(1)} = \begin{bmatrix} 0.5 \\ 2 \end{bmatrix} (2) + \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

$$h = \text{ReLU}(z^{(1)}) = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

$$\mathbf{y_2} = \begin{bmatrix} -1 & 0.5 \end{bmatrix} \begin{bmatrix} 2 \\ 2 \end{bmatrix} + 0 = -1$$

Model 3

$$W^{(1)} = \begin{bmatrix} -2 \\ 1 \end{bmatrix}, b^{(1)} = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$$

$$W^{(2)} = \begin{bmatrix} 1 & 2 \end{bmatrix}, b^{(2)} = 1$$

$$z^{(1)} = \begin{bmatrix} -2 \\ 1 \end{bmatrix} (2) + \begin{bmatrix} 2 \\ 0 \end{bmatrix} = \begin{bmatrix} -2 \\ 2 \end{bmatrix}$$

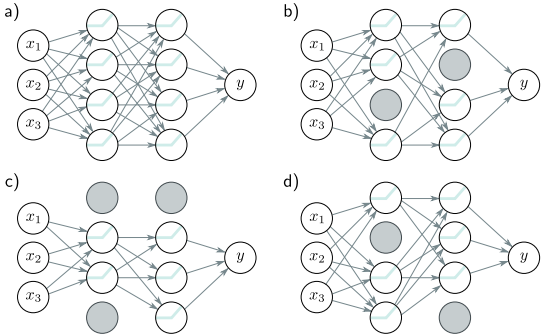
$$h = \text{ReLU}(z^{(1)}) = \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

$$\mathbf{y_3} = \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 2 \end{bmatrix} + 1 = 5$$

Ensemble Prediction: $\hat{y}_{ens} = \frac{y_1 + y_2 + y_3}{3} = \frac{7}{3}$

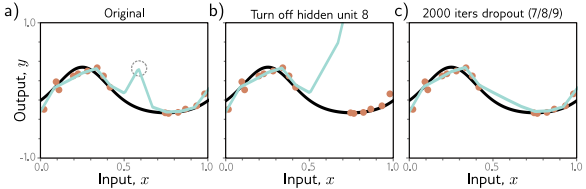
Dropout

- Clamps a random subset (e.g., 50%) of hidden units to **zero** at each SGD iteration.
- Encourages the network to be robust; it cannot depend too heavily on any single hidden unit.
- Encourages weights to have smaller magnitudes.
- Inference: Use the *weight scaling rule* multiply weights by $1 - P_{dropout}$.



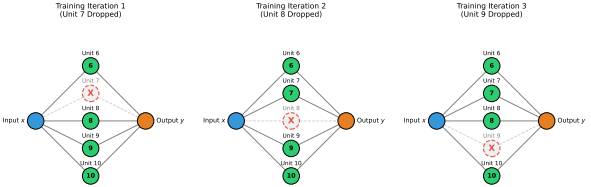
Dropout Mechanism

- Without dropout, multiple units can sequentially "conspire" to produce undesirable local kinks in the function to fit noise perfectly.
- If one unit is removed (as in dropout), the output changes drastically, penalizing the loss.
- Subsequent gradient steps compensate by smoothing the function, naturally eliminating unnecessary kinks over time.

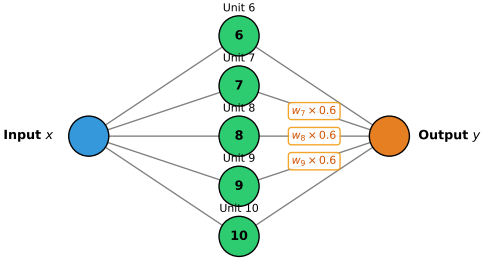


Dropout Mechanism: Training vs. Inference

Training Stage: At each iteration, a random subset of hidden units is set to zero with probability $P_{dropout} = 0.4$.



Inference Stage: All hidden units are active. To compensate for the extra active units, we multiply the downstream weights of the dropped units by $1 - P_{dropout} = 0.6$.



Dropout Inference: Numerical Example

Problem: Calculate the final network output during inference given:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}, \quad p = 0.3$$

Dropout Inference: Numerical Example

Problem: Calculate the final network output during inference given:

$$\mathbf{x} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad W^{(1)} = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}, \quad p = 0.3$$

1. Compute hidden layer activations (assuming a linear activation function for simplicity):

$$\mathbf{h} = W^{(1)}\mathbf{x} = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \end{bmatrix}$$

2. Apply the Weight Scaling:

$$W^{(2)}_{scaled} = (1 - 0.3) \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 3.5 \\ 4.2 \end{bmatrix}$$

3. Compute final output

$$y = \left(W^{(2)}_{scaled} \right)^T \mathbf{h} = \begin{bmatrix} 3.5 & 4.2 \end{bmatrix} \begin{bmatrix} 4 \\ 6 \end{bmatrix} = 39.2$$

Applying Noise

- **Noise to Inputs:** Smooths out the learned function (equivalent to regularizing input derivatives).
- **Noise to Weights:** Encourages optimization to find wide, flat, and stable local minima.
- **Label Smoothing:** Discourages overconfident predictions by assuming a proportion ρ of training labels are incorrect.

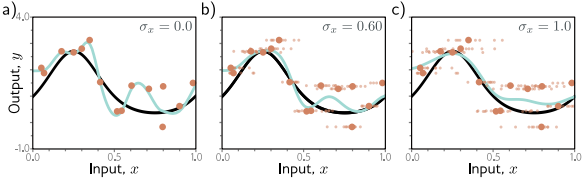


Figure: Applying noise to the inputs.

Bayesian Inference

- Standard ML approaches are often overconfident, picking a single best parameter set.
- **Bayesian Approach:** Treats parameters as unknown variables and computes a posterior distribution over them:

$$Pr(\phi|\{\mathbf{x}_i, \mathbf{y}_i\}) = \frac{\prod_{i=1}^{I} Pr(\mathbf{y}_i|\mathbf{x}_i, \phi)Pr(\phi)}{\int \prod_{i=1}^{I} Pr(\mathbf{y}_i|\mathbf{x}_i, \phi)Pr(\phi)d\phi}$$

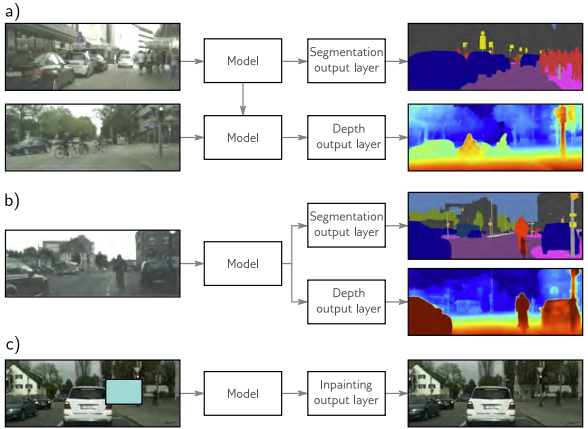
- Inference becomes an infinite weighted sum (integral) over all possible models:

$$Pr(\mathbf{y}|\mathbf{x}, \{\mathbf{x}_i, \mathbf{y}_i\}) = \int Pr(\mathbf{y}|\mathbf{x}, \phi)Pr(\phi|\{\mathbf{x}_i, \mathbf{y}_i\})d\phi$$

- **Challenge:** Analytically intractable for deep networks; requires heavy approximations (e.g., variational inference).

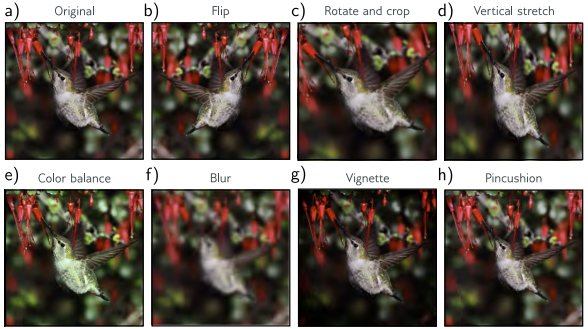
Transfer, Multi-task, and Self-supervised Learning

- **Transfer Learning:** Pre-train on a plentiful secondary task, then fine-tune parameters for a related task with limited data.
- **Multi-task Learning:** Train the network to solve multiple related problems concurrently (e.g., segmentation and depth estimation).
- **Self-supervised Learning:** Generates labels from the input itself:
  - *Generative:* Inpaint or predict masked parts of data.
  - *Contrastive:* Learn to pair transformed versions of the same instance.



Data Augmentation

- **Goal:** Expand the dataset by applying transformations to the input data that do not alter the semantic label.
- **Examples for Images:** Rotation, horizontal flipping, blurring, altering color balance or adding vignette.
- **Examples for Text:** Synonym substitution, back-translation.
- Teaches the model *invariance* to irrelevant transformations, heavily restricting the effective capacity of the network to memorize noise.



Summary of Regularization

Regularization techniques improve generalization across four main mechanisms:

1. **Make function smoother:** L2 regularization, Early stopping.
2. **Increase data:** Data augmentation, Transfer/Multi-task learning.
3. **Combine multiple models:** Ensembling, Dropout, Bayesian approaches.
4. **Find wider minima:** Implicit SGD regularization, Adding noise to weights.